Sistemas Distribuídos · Trabalho 2

Fintech SD
microsserviços com API Gateway e gRPC

Evolução da fintech de pagamentos do Trabalho 1: frontend web, API Gateway FastAPI com JWT, dois microsserviços gRPC em linguagens diferentes (Java e Python) e PostgreSQL 18 no Cloud SQL.

Frontend HTML/JS | nginx · HTTPS | FastAPI · JWT · Pydantic | gRPC · Protobuf | Java 21 | Python 3.12 | Cloud SQL · PostgreSQL 18

Equipe: Henry Engels de Aguiar · Paulo Francisco Oliveira Teles · Matheus Santana de Moura

Código: `github.com/henryengels/Trabalho2-sd`

Do Trabalho 1 ao Trabalho 2

A lógica de negócio da fintech foi mantida; a infraestrutura distribuída ao redor dela foi refeita.

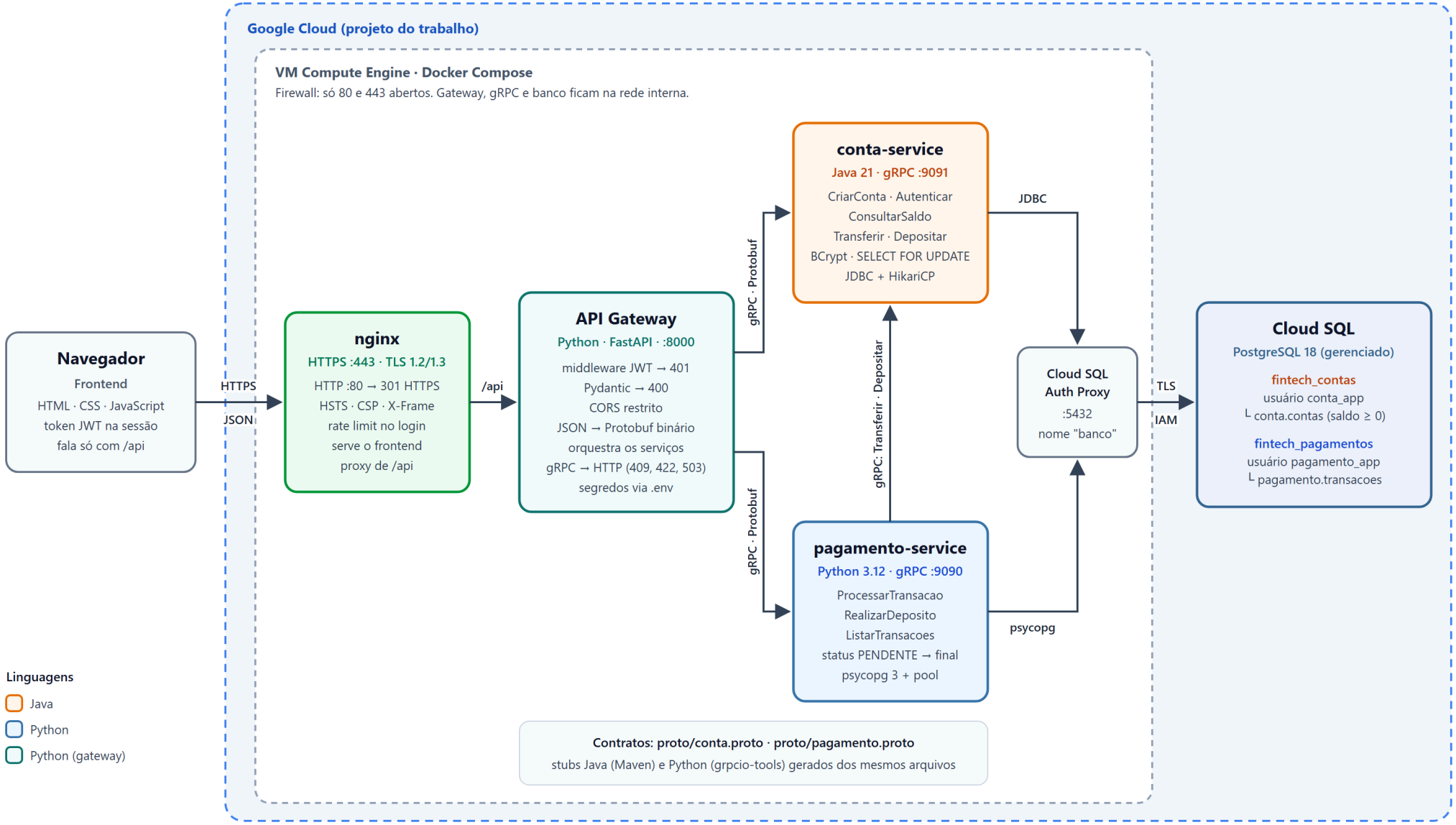
Trabalho 1

- Cliente de terminal falando gRPC direto com o serviço
- pagamento-service e conta-service, ambos em Java
- Saldos num `HashMap` em memória (perdidos ao reiniciar)
- Concorrência com `synchronized` no processo
- Host e portas fixos no código, gRPC sem autenticação

Trabalho 2

- Frontend web via **nginx com HTTPS**
- **API Gateway FastAPI**: ponto único de entrada, JWT, Pydantic, CORS
- conta-service em **Java** e pagamento-service em **Python**
- **PostgreSQL 18** (Cloud SQL), com um banco por microsserviço
- Atomicidade com transação SQL e `SELECT … FOR UPDATE`
- Configuração por `.env`, implantação com Docker Compose na GCP

Arquitetura de microsserviços



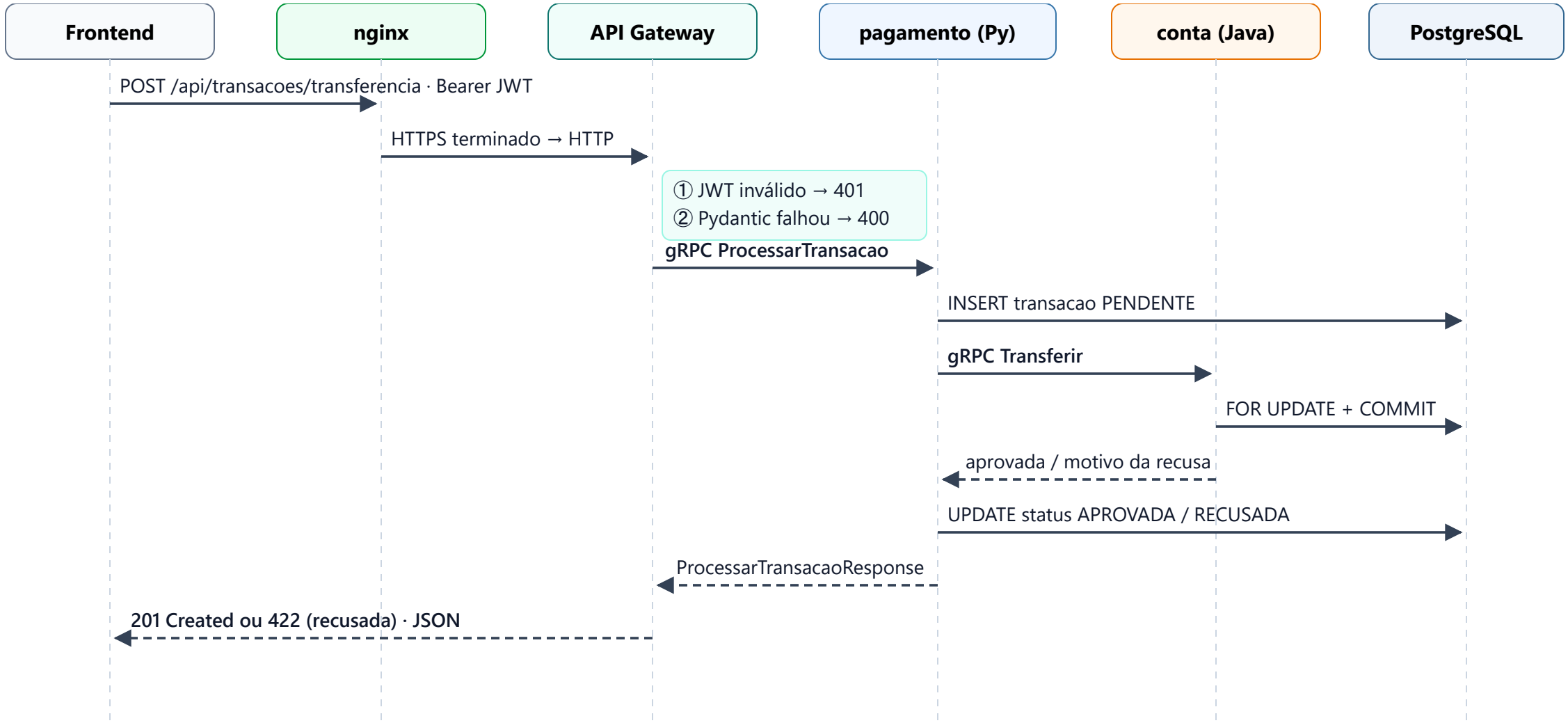
Componentes e tecnologias

Somente o nginx é exposto. O restante se comunica pela rede interna do Docker Compose.

COMPONENTE	TECNOLOGIA	PORTA	RESPONSABILIDADE
Frontend	HTML, CSS, JavaScript	—	Login, cadastro, saldo, depósito, transferência e extrato; chama apenas `/api`
nginx	nginx 1.27	80 / 443	HTTPS, redirecionamento 80→443, cabeçalhos de segurança, rate limit, proxy para o gateway
API Gateway	Python · FastAPI · PyJWT · grpc.aio	8000	Middleware JWT, validação Pydantic, CORS, tradução JSON ↔ Protobuf e orquestração
conta-service	Java 21 · grpc-java · JDBC · HikariCP · BCrypt	9091	Contas e credenciais; único serviço que altera saldos
pagamento-service	Python 3.12 · grpcio · psycopg 3	9090	Registra cada transação, delega ao conta-service e fornece o extrato
Cloud SQL	PostgreSQL 18 + Cloud SQL Auth Proxy	5432	Bancos `fintech_contas` e `fintech_pagamentos`, um por serviço; TLS com autorização IAM

Fluxo de uma transferência

JSON na borda, Protobuf binário entre os serviços, transação SQL no banco.



Contratos gRPC: um .proto, duas linguagens

Os stubs Java (Maven + protoc) e Python (grpcio-tools) são gerados dos mesmos arquivos em `proto/`.

```
// proto/conta.proto — implementado em Java
service ContaService {
  rpc CriarConta(CriarContaRequest) returns (CriarContaResponse);
  rpc Autenticar(AutenticarRequest) returns (AutenticarResponse);
  rpc ConsultarSaldo(ConsultarSaldoRequest) returns (...);
  rpc Transferir(TransferirRequest) returns (TransferirResponse);
  rpc Depositar(DepositarRequest) returns (DepositarResponse);
}
message TransferirRequest {
  string conta_origem = 1;  string conta_destino = 2;
  int64  valor_centavos = 3;  // dinheiro em centavos
}
```

```
// proto/pagamento.proto — implementado em Python
service PagamentoService {
  rpc ProcessarTransacao(ProcessarTransacaoRequest) returns (...);
  rpc RealizarDeposito(RealizarDepositoRequest) returns (...);
  rpc ListarTransacoes(ListarTransacoesRequest) returns (...);
}
```

Python → Java

```
# pagamento-service/pagamento/cliente_contas.py
request = conta_pb2.TransferirRequest(
    conta_origem=origem, conta_destino=destino,
    valor_centavos=valor)
return self._stub.Transferir(request, timeout=5)
```

Erros padronizados

Os serviços usam os códigos gRPC, e o gateway os converte:
`INVALID_ARGUMENT`→400 · `UNAUTHENTICATED`→401 · `ALREADY_EXISTS`→409 · `UNAVAILABLE/DEADLINE_EXCEEDED`→503

API Gateway (FastAPI): rotas e respostas

Recebe JSON, autentica, valida, serializa em Protobuf e despacha por gRPC.

ROTA	JWT	SERVIÇO GRPC	SUCESSO
`POST /api/auth/registro`	—	CriarConta	201
`POST /api/auth/login`	—	Autenticar → JWT	200
`GET /api/contas/me`	✓	ConsultarSaldo	200
`GET /api/contas/me/extrato`	✓	ConsultarSaldo ‖ ListarTransacoes	200
`POST /api/transacoes/deposito`	✓	RealizarDeposito	201
`POST /api/transacoes/transferencia`	✓	ProcessarTransacao	201

400 JSON malformado, campo ausente, formato ou valor inválido, campo extra

401 sem token, token inválido ou expirado (barrado no middleware)

409 conta já cadastrada · **422** saldo insuficiente

503 microsserviço sem resposta

```
POST /api/transacoes/deposito {"valor": 10.555}
→ 400 Bad Request
{"detail": "Dados inválidos.",
 "erros": [{"campo": "valor",
   "mensagem": "Use no máximo 2 casas decimais."}]}
```

Segurança

Defesa em camadas: borda, gateway, serviços e banco.

HTTPS e proxy

nginx com TLS 1.2/1.3, 80→443, HSTS, CSP, X-Frame-Options, rate limit no login, corpo limitado a 16 KB.

JWT

Emitido no login (HS256, emissor e expiração obrigatórios). Middleware valida o `Authorization: Bearer` em toda rota protegida → 401.

Autorização

A conta de origem vem do token. Enviar `conta_origem` no corpo é recusado (400): ninguém movimenta a conta de outro.

Validação Pydantic

Campos obrigatórios, padrões, limites, até 2 casas decimais, `extra="forbid"`. Os serviços revalidam a entrada.

CORS e segredos

CORS só para as origens do `.env`. Segredos (`JWT_SEGREDO` e senhas do banco) no `.env`, fora do Git e das imagens.

Dados

Senhas com BCrypt, SQL parametrizado, um usuário por serviço sem acesso ao banco do outro, Cloud SQL via Auth Proxy (TLS + IAM), contêineres sem root.

Banco de dados: PostgreSQL 18 no Cloud SQL

Relacional e ACID: débito e crédito acontecem juntos ou não acontecem.

BANCO · USUÁRIO	SERVIÇO	TABELA
`fintech_contas` `conta_app`	conta-service	`conta.contas`: titular, hash BCrypt, saldo `CHECK (≥ 0)`
`fintech_pagamentos` `pagamento_app`	pagamento-service	`pagamento.transacoes`: UUID, tipo, origem, destino, valor, status

- **Database-per-service**: cada usuário só se conecta ao próprio banco
- Bancos e usuários criados pelo contêiner `preparar-banco`
- Nenhum dado em memória ou simulado: tudo é lido e gravado no banco
- Transação gravada como `PENDENTE` e atualizada após o gRPC

```
-- conta-service: transferência atômica
SELECT id, saldo_centavos FROM conta.contas
 WHERE id IN (?, ?) ORDER BY id FOR UPDATE;
-- valida existência e saldo
UPDATE conta.contas SET saldo_centavos = saldo_centavos - ? ...
UPDATE conta.contas SET saldo_centavos = saldo_centavos + ? ...
COMMIT;
```

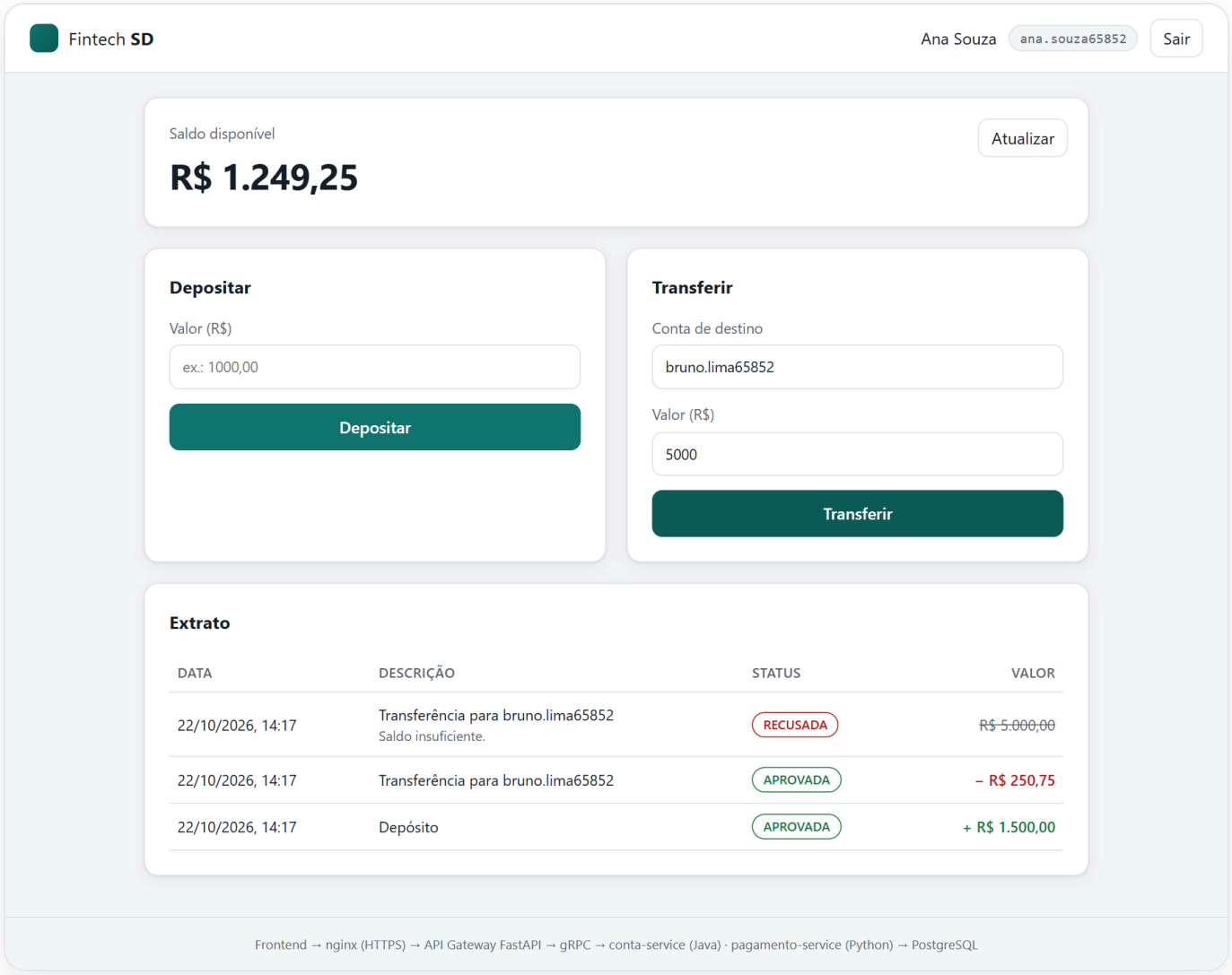
Teste de concorrência

20 transferências simultâneas de R$ 100,00 com saldo de R$ 849,50:

8 aprovadas · **12 recusadas** · **soma dos saldos inalterada**

Interface do usuário

Erros de validação (400) aparecem em cada campo; transações recusadas (422) ficam no extrato.





Implantação na Google Cloud e testes

Mesmo `docker-compose.yml` nos dois ambientes; muda apenas o perfil do banco no `.env`.

Google Cloud

- **Cloud SQL** PostgreSQL 18: `fintech_contas` e `fintech_pagamentos`
- **VM Compute Engine** com Docker Compose (`COMPOSE_PROFILES=gcp`)
- **Cloud SQL Auth Proxy** com o papel IAM *Cloud SQL Client*
- Bancos e usuários criados pelo `preparar-banco`
- Firewall liberando somente `tcp:80,443`
- Passo a passo com `gcloud` em `docs/gcp.md`

Teste de ponta a ponta

`bash scripts/testar-api.sh https://IP`

- **401** sem token / token inválido
- **400** campos ausentes, formato inválido, JSON malformado
- **201** cadastro, depósito e transferência
- **409** conta duplicada · **422** saldo insuficiente
- **200** login, saldo e extrato

22 verificações, 0 falhas

Código-fonte

https://github.com/henryengels/Trabalho2-sd

Frontend

`frontend/` e `nginx/`

API Gateway

`api-gateway/` (FastAPI)

Contratos

`proto/conta.proto`
`proto/pagamento.proto`

conta-service

`conta-service/` (Java 21)

pagamento-service

`pagamento-service/` (Python)

Execução

`docker compose up -d --build`
guias em `docs/`